TEMA 1 – Búsqueda no informada

→ **TREE-SEARCH:** solo lista **OPEN** (re-expande nodos ya explorados → gasta memoria y tiempo)

→ **GRAPH-SEARCH:** lista **OPEN** y **CLOSED** → control de estados repetidos (+memoria pero -tiempo)

Criterio	Anchura f(n)=nivel(n)	Coste uniforme f(n)=g(n)	Prof. Tree-Search f(n)=-nivel(n)	Prof. Iterativa
Temporal	$O(b^{d+1})$	$O(b^{C^*/\varepsilon})$	$O(b^m)$	$O(b^d)$
Espacial	$O(b^{d+1})$	$O(b^{C^*/\varepsilon})$	$O(b \cdot m)$	$O(b \cdot d)$
Optima?	Sí *	Sí	No	Sí *
Completa?	Sí	Sí **	No	Sí

* Óptima si los costes de las acciones son todos iguales

** Completa si los costes de las acciones son $\geq \varepsilon$ para un ε positivo

b *factor medio de ramificación*
d *nivel medio de profundidad*
C* *coste solución óptima*
m *nivel máximo profundidad*

Coste espacial

A > Voraz > Anchura > C.U. > Prof. > P.I.* = IDA*

Coste temporal

Prof. = Voraz > Anchura > P.I. = IDA > C.U. = A**

TEMA 2 – Búsqueda con heurísticas

BÚSQUEDA VORAZ → f(n) = h(n)

- **Temporal = espacial =** $O(b^m)$
- **Completa:** solo GRAPH-SEARCH
- **Óptima:** No

BÚSQUEDA A* → f(n) = g(n) + h(n)

- **Temporal =** $O(b^{C*/\min_coste_acción})$
- **Espacial:** mantiene todos los nodos en memoria (su mayor problema)
- **Completa:** Sí
- **Óptima:**
  - **TREE-SEARCH:** si h(n) es admisible
  - **GRAPH-SEARCH con re-expansión:** si h(n) es admisible
  - **GRAPH-SEARCH sin re-expansión:** h(n) admisible y consistente

Heurística admisible: si nunca sobreestima el coste real de llegar al objetivo: ∀ **n, h(n) ≤ h*(n)**

- ***Heurística piezas descolocadas:*** *admisible*
- ***Heurística distancia de Manhattan:*** *no admisible (solo se pueden mover las piezas en horizontal y vertical, no en diagonal).*

Heurística consistente: h(n) ≤ h(n') + coste de llegar a n. n' es el nodo siguiente a n. Se ha de cumplir para todo n.

h(n) *función heurística*
g(n) *coste de llegar a un nodo*

h(n) = 0 ausencia conocimiento
h(n) = h*(n) conocimiento máximo
h*(nodo_inicial) = coste mínimo para llegar al nodo final

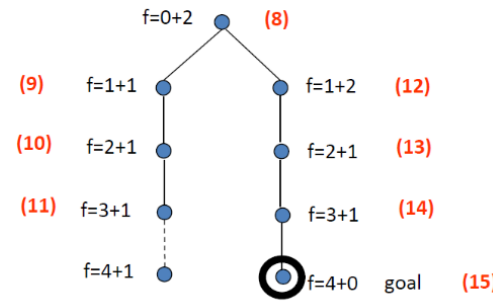
Si h2(n) ≥ h1(n) ∀ n, entonces **h2 domina a h1** → h2 nunca expandirá más nodos que h1

TEMA 3 – Búsqueda heurística con memoria limitada

ALGORITMO A* DE PROFUNDIZACIÓN ITERATIVA (IDA*): Igual que Profundización Iterativa (ID) pero el límite es el **valor-f (g+h)**.

Ventaja: usa menos memoria que A*. **Desventaja:** no comprueba estados repetidos, genera muchos más nodos que A*

- **Temporal =** $O(b^d)$
- **Espacial =** $O(b \cdot d)$ usa poca memoria
- **Óptima:** bajo las mismas condiciones que A*



ALGORITMO RECURSIVE BEST FIRST SEARCH (RBFS): parecido a Profundidad (TREE-SEARCH) con *backtracking*, pero en lugar de hacer *backtracking* cronológico recuerda el **valor-f** del mejor camino alternativo disponible.

→ *Si el nodo de la rama que estamos explorando supera el **valor-f**, se olvida el sub-árbol actual y vuelve al camino alternativo.*

→ **Óptimo:** mismas condiciones que A* (heurística admisible)

WOOLAH

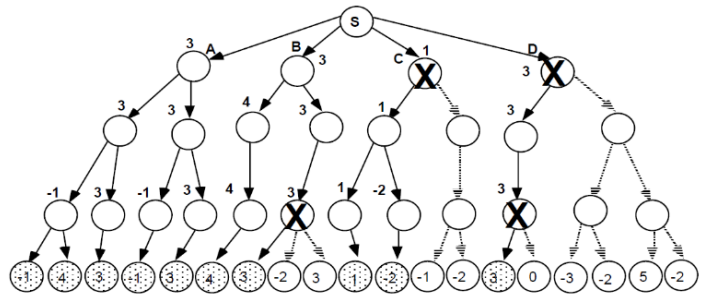
TEMA 4 – Búsqueda con adversario

ALGORITMO ALFA-BETA

Nodos MAX valor α
Nodos MIN valor β

Corte beta: *α provisional $\geq \beta$ predecesor (no sólo padre)*

Corte alfa: *β provisional $\leq \alpha$ predecesor (no sólo padre)*



TEMA 5 – Sistemas basados en reglas (SBR)

Base de reglas: contiene reglas que representan como cambia el mundo (***conocimiento procedural***)
Base de hechos: contiene hechos que representan el estado del mundo (***conocimiento declarativo***)
Motor de inferencia: mecanismo de control de un SBR (procesa la información de la BR y BH)

DEFINIR HECHOS CON CLIPS:

(deffacts <deffacts-name> [<comment>] <fact>*)

1	2	4
7	8	6
3		5

```
(deffacts puzzle_Rep1
    (puzzle 1 2 4 7 8 6 3 0 5)}

(deffacts puzzle_Rep2
    (puzzle 1 1 1)
    (puzzle 1 2 2)
    (puzzle 1 3 4)
    (puzzle 2 1 7)
    ...)
```

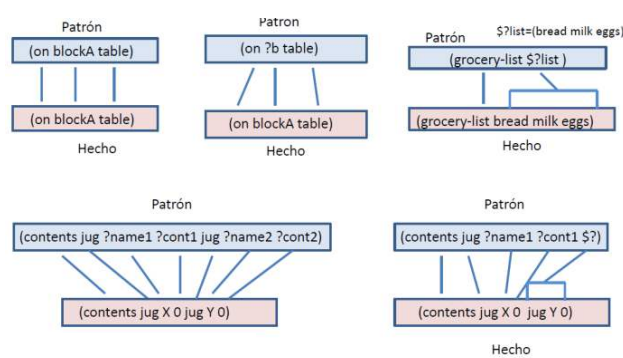
DEFINIR REGLAS CON CLIPS:

```
(defrule <rule name> ["comment"]
   <conditional-element>*        ; left-hand side (LHS) de la regla
                                 ; condiciones a satisfacer
=>
   <actions>*      ;; right-hand side (RHS) de la regla
                   ;; acciones a ejecutar cuando la regla se dispara (cuando la regla
                   ;; se selecciona para ser ejecutada)
)
```

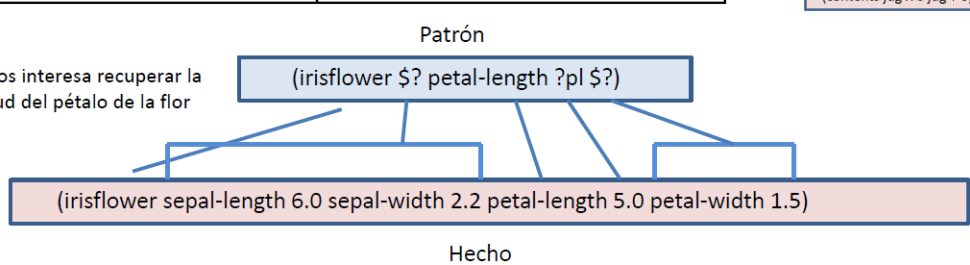
- **assert**: añade un hecho en la BH **(assert <fact>+)**
- **retract**: borra un hecho en la BH **(retract <fact-index>+)**
- **printout**: Imprime algo por pantalla

PATTERN MATCHING (unificación de patrones)

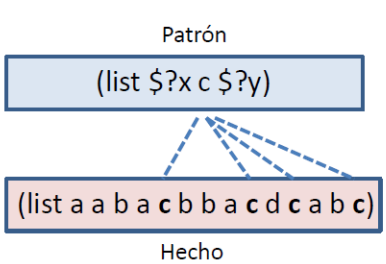
Elemento del patrón	Elemento del hecho
Constante	Constante
Variable mono-valuada (?x)	Un único valor
Variable multi-valuada ($?x)	Cero o más valores
Comodín mono-valuado (?)	Un único valor sin mantener la ligadura
Comodín multi-valuado ($?)	Cero o más valores sin mantener la ligadura



Solo nos interesa recuperar la longitud del pétalo de la flor



Cuando dos o más variables multi-valuadas aparecen en un patrón, el patrón y el hecho pueden unificar (match) más de una vez



#Match	$?x	$?y
1	(a a b a c b b a c d c a b)	()
2	(a a b a c b b a c d)	(a b c)
3	(a a b a c b b a)	(d c a b c)
4	(a a b a)	(b b a c d c a b c)

Unificación de reglas

Lag parte izquierda de las reglas unifica cuando existe al menos un hecho en la BH que **unifica** con cada patrón de la regla y todos los tests de la regla se evalúan a TRUE.

Cuando una regla unifica, se dice que se ha encontrado una ***instancia de la regla***.

Puede haber más de una unificación (instancia) para la misma regla dependiendo del número de hechos que unifiquen con cada patrón.